

STOCK PREDICTOR PRO - SESSION HANDOFF

File name: upgrade.pdf | Repo: StockPredictorApp - Copy | Branch: main

Commit: 2193066 - Make demo results JSON-serializable and precompute the full demo set (prior: afe19c0)

Session: offline precompute of the public /demo cache succeeded 18/18; two build_demo_result bugs fixed so the cache writes clean JSON.

0. HOW TO CONTINUE TOMORROW

When the user says 'read and do upgrade pdf': open this PDF, run Section 6 (NEXT STEPS TOP-DOWN) in order, and report results. Everything below is the state this session reached.

1. OBJECTIVE

1 Fix a production out-of-memory (OOM) crash on the authenticated /analyze route: a live 100-epoch AUTO LSTM/GRU training run (~78k parameters) exceeded the Render free-tier worker's 512MB budget and gunicorn OOM-killed the process mid-request. DONE in afe19c0.

2 Lock the public unauthenticated /demo route so it ONLY serves precomputed cached results: no live training, no free-text ticker input, out-of-set tickers rejected with a 400. DONE in afe19c0.

3 THIS SESSION: make the offline precompute actually work (build_demo_result crashed on every ticker: a NameError and a JSON key crash), then precompute all 18 demo tickers and ship the JSON so the visible demo has data on a fresh deploy.

2. UNDERSTANDING OF DEPLOYMENT (facts confirmed)

1 The LIVE app is the refactored package: wsgi.py -> stockpredictor.create_app; Dockerfile CMD runs gunicorn -c gunicorn.conf.py wsgi:app. The root monolith app.py is NOT deployed (has no /demo route).

2 Single worker (eventlet). gunicorn.conf.py refuses WEB_CONCURRENCY>1 without REDIS_URL. render.yaml: free plan, PUBLIC_DEMO=true, WEB_CONCURRENCY=1, ENABLE_ALERT_SCHEDULER=false.

3 Dockerfile chowns /app to the non-root appuser, so /app/data_cache and /app/runtime_state stay writable.

4 production_core.py: BASE_DIR, CACHE_DIR = BASE_DIR/'data_cache', STATE_DIR = BASE_DIR/'runtime_state', sanitize_symbol, cached(), json_endpoint, install_request_guards, CSRF handling.

5 data_cache/ is gitignored EXCEPT data_cache/demo_precomputed/** (exception added this session) so the 18 precomputed JSONs ship inside the image via Dockerfile COPY . . - without this /demo/analyze would 404 on a fresh deploy.

6 do NOT touch in follow-ups: auth, OTP, Resend email, CSRF/session code. tests.test_stockpredictor must keep passing.

3. WHAT WAS DONE

3.1 Previous session (commit afe19c0) - recap

1 predictor_core.py: train() epoch/batch/ES defaults now env-driven (TRAIN_EPOCHS=20, TRAIN_BATCH_SIZE=32, EARLY_STOPPING_PATIENCE=5); OOM guard wraps build+fit (release_tf_memory() then raise ModelResourceError('Analysis temporarily unavailable, please try again.')); generate_forecast_report() re-raises ModelResourceError instead of swallowing it into the fake-data fallback.

2 stockpredictor/services/jobs.py: JobManager has a threading.Semaphore(1) training gate (GATE_WAIT_SECONDS=30, BUSY_MESSAGE 'Another analysis is already in progress...'); _run() finally calls _free_model_memory() so exactly one training job runs at a time and TF memory is freed between jobs.

3 NEW stockpredictor/services/demo_cache.py: DEMO_TICKERS (18: AAPL MSFT TSLA GOOGL AMZN NVDA META NFLX AMD JPM V DIS SPY QQQ RELIANCE.NS TCS.NS HDFCBANK.NS INFY.NS); DEMO_CACHE_DIR = DEMO_CACHE_DIR env or CACHE_DIR/'demo_precomputed'; API list_demo_tickers / demo_cache_path / get_demo_result (KeyError out-of-set, FileNotFoundError missing) / demo_write (atomic tmp+replace) / build_demo_result (offline-only full pipeline).

4 NEW scripts/precompute_demo_cache.py: standalone offline runner (--tickers / --days / --period / --risk / --model-type / --workers); validates against the demo set (exit 2), per-ticker try/except, atomic overwrite, exit 1 if any ticker failed.

5 stockpredictor/views/dashboard.py: demo() passes demo_tickers; NEW /demo/analyze (GET+POST, unauthenticated): whitelist 400 'not part of the public demo set', get_demo_result() -> demo_results.html, FileNotFoundError -> 404 'not available yet'. Never trains, never free-text.

6 templates: dashboard.html 'Try a Precomputed Analysis' card (demo_only); NEW demo_results.html public read-only results page (forecast chart/table, technical, regime/model, sentiment, analyst report, price history, fundamentals, switch-symbol dropdown, register CTA; all authenticated widgets removed).

7 config/docs/tests: Settings TRAIN_EPOCHS / TRAIN_BATCH_SIZE / EARLY_STOPPING_PATIENCE; .env.example + README; tests/test_demo_cache.py (13 tests).

3.2 This session (commit 2193066) - two build_demo_result bugs fixed

1 `NameError 'detect_regime' / 'rule_based_summary' is not defined: the imports lived in build_demo_result's LOCAL scope but _demo_enrich() referenced them in its own scope. FIX: moved 'from ..services.llm_report import rule_based_summary' and 'from ..services.regime import auto_select_model, detect_regime' into _demo_enrich().` Light imports (numpy only) - tests never pull in TensorFlow.

2 JSON crash 'keys must be str, int, float, bool or None, not Timestamp': `json.dump(default=str)` never applies the hook to dict KEYS; `fundamentals.financial_statements.{income_statement,balance_sheet,cash_flow}` arrive keyed by pandas Timestamp. FIX: `_coerce_keys()` helper recursively converts non-basestring keys to isoformat strings and wraps BOTH return paths of `build_demo_result()`. The ymd template filter (`str(value)[:10]`) renders the ISO keys identically to the live app.

3 Shippability: `.gitignore` now tracks `data_cache/demo_precomputed/**` (pattern `data_cache/*` + negations) so the JSONs are committed and ship via Docker COPY `..`; README updated ('commits nothing' -> precomputed results are committed so fresh deploys ship the demo). Secret scan of all 18 JSONs: NONE.

4 Tests: `JsonKeyCoercionTests` added (2 tests) - flatten Timestamp keys to ISO strings + prove `json.dumps` safety.

3.3 Demo data (this session)

1 Smoke test AAPL+MSFT with the fixed code: first pass exposed the two bugs above; after the fix 2/2 cached.

2 The full 18-ticker run hit a transient Yahoo Finance DNS outage ('Could not resolve host: query2.finance.yahoo.com' ~30s), failing 14/18. Retried those 14 (Test-NetConnection query2.finance.yahoo.com:443 = True) - all 18/18 now cached. GOOGL was re-run once more to backfill fundamentals that a provider hiccup dropped mid-run.

3 Every cached JSON: `report.forecast_rows` (30 rows), `report.forecast_bands`, `price_history` (~90 points), `technical`, `sentiment`, `fundamentals` (`company_info` + `financial_statements` x3 tables), `regime`, `model_selection`, `analyst_report`, `source=precomputed`, `generated_at`, `params`. No top-level error key.

4. VERIFICATION RESULTS

1 `python -m py_compile` on all changed files: PASS.

2 & `.\.venv310\Scripts\python.exe -m unittest tests.test_demo_cache -v: 15 OK (~12s)` (13 original + 2 `JsonKeyCoercion`).

3 & `.\.venv310\Scripts\python.exe -m unittest tests.test_stockpredictor: 33 OK (~17s)` - `auth/OTP/CSRF/demo/email` untouched.

4 All 18 demo JSONs validated programmatically: `rows=30` each, `error=None`, `source=precomputed`, `price_hist=90`. `BAD=0`.

5 Test-client render probe (`create_app TESTING + PUBLIC_DEMO=true`, CSRF off against real precomputed cache): `/demo -> 200` with all 18 `<option>`; `/demo/analyze?symbol=AAPL / RELIANCE.NS / INFY.NS -> 200` with the 'precomputed demo' chip and 31 `<tr>`; `symbol=FAKE GET -> 400 'not part of the public demo set'`; `POST symbol=NOPE -> 400`.

6 NOTE: `/demo`'s card grid still calls `_fetch_live` for quote snippets (22-record fetches seen in probe logs). That is pre-existing dashboard behavior, network-only, no training, OOM-safe.

5. GIT STATE

1 Committed: 2193066 'Make demo results JSON-serializable and precompute the full demo set' (22 files: 18 demo JSONs + `demo_cache.py` bugfix + 2 tests + `.gitignore` + README). Branch main.

2 Working tree clean at commit time except this `upgrade.pdf` (intentionally left untracked, matching the previous session).

3 Branch is 2 commits ahead of origin/main (afe19c0 + 2193066) - push triggers the Render redeploy.

6. NEXT STEPS (TODO - continue from here tomorrow)

1 DEPLOY (deploy-time, needs Render account or docker): push main (or manual docker build) so the 18 committed JSONs ship in the image; then visit `/demo -> 'Try a Precomputed Analysis' -> AAPL`. The page must render instantly from JSON with the 'precomputed demo' chip.

2 DEPLOY CHECKLIST: (a) `/demo` shows the 18-ticker dropdown; (b) `/demo/analyze?symbol=AAPL -> 200` instant; (c) `/demo/analyze?symbol=FAKE -> 400 'not part of the public demo set'`; (d) every demo ticker now has its JSON so no 404 should occur (MSFT etc. all cached); (e) grep logs for 'Fetching data and training' - must only ever come from authenticated `/analyze`, never `/demo`.

3 MEMORY VERIFY (after deploy): (a) authenticated `/analyze` logs 'Epoch 1/20' (not /100); watch Render memory; lower `TRAIN_EPOCHS` to 10 if still high. (b) Fire two simultaneous `/analyze`: the second must answer 'Another analysis is already in progress' (after ~30s queue), NOT double memory. (c) Watch for 'Out-of-memory while generating forecast'; if seen lower `TRAIN_EPOCHS` / `TRAIN_BATCH_SIZE`. (d) 'Analysis temporarily unavailable, please try again.' renders as a clean error page, not a 502.

4 OPTIONAL (NOT STARTED, task allowed as levers): expose the batch/lookback lever in the UI (AdvancedStockPredictor `lookback_days=60` default unchanged; not exposed). Weekly demo-cache refresh: re-run `scripts/precompute_demo_cache.py` offline and commit the regenerated JSONs.

5 AT END OF NEXT SESSION: regenerate this `upgrade.pdf` so it always reflects the newest state, and commit any new work.

6 Time note (heads-up): the 18-ticker precompute took ~50 min total here (~1-3 min/ticker with cached models; real training where models were missing, early-stopped at ~6 epochs). A weekly refresh is a background task, not an on-page request.

7. ENVIRONMENT / COMMANDS REFERENCE

OS : Windows, shell = PowerShell 5.1. Use Get-ChildItem / Get-Content, not ls/cat. No && : chain with ';' or if (\$?) { ... }.

Python : .venv310 (Python 3.10.11, TensorFlow 2.15.0). System python.exe is 3.14 (has pymupdf - used to read/write this PDF). Use the venv for tests:

```
& .\venv310\Scripts\python.exe -m unittest tests.test_demo_cache
```

```
& .\venv310\Scripts\python.exe -m unittest tests.test_stockpredictor
```

discover: & .\venv310\Scripts\python.exe -m unittest discover (slow; skips live API tests unless RUN_EXTERNAL_TESTS=1 - prefer module-by-module)

precomp : & .\venv310\Scripts\python.exe scripts/precompute_demo_cache.py [--tickers AAPL MSFT] [--days 30 --period 1y --risk medium --model-type AUTO]. Exit 0 all ok, 1 any fail, 2 unknown ticker. Writes data_cache/demo_precomputed/<SYM>.json (now tracked).

wsgi.py : re-launches under .venv310 when present.

gotcha : the Grep tool's path filter is unreliable; use include/pattern carefully or fall back to ripgrep (rg) / Select-String.

grep : model training load lives in predictor_core.py; heavy imports stay lazy in stockpredictor/services/stocks.py (_load_modules).

8. DANGER / DO-NOT-BREAK

1 NEVER restore the 100-epoch default; NEVER let /demo call predict_forecast / train / generate_forecast_report (OOM-kills the free worker).

2 Keep /demo strictly read-only and unauthenticated, but the fixed 18-ticker list is NOT optional - it is the lockdown itself.

3 Keep green: tests.test_stockpredictor (33) + tests.test_demo_cache (15).

4 Auth/OTP/Resend/CSRF/session code was not touched and must not be changed in follow-ups.

5 data_cache/demo_precomputed/** is intentional committed content; do NOT commit the rest of data_cache (yfinance/opt_raw CSV + quote/search/sentiment caches stay ignored), and do NOT hand-edit the JSONs with secret-like content.